

- 1 软件体系结构中有什么活动, qqqqqq 还差输入输出?
- 2 产品线
- 3 软件设计策略 strategies ffffffffff
- 4 如何建模质量属性场景 ffffffffff
- 5 ATAM 过程的每个阶段生成的输出 ffffffffff
- 6 描述 组件-连接件风格、allocation styles 分配风格、module styles
- 7 什么是 ASRs ffffffffff
- 8 面向对象原则(不是设计原则)
- 9 什么该被包含, 在一个典型的体系结构文档中 ffffffffff
- 10 描述"4+1"Views qqqqqq 不够完整?
- 11 软件设计中变化的可能维度, 推迟绑定时间
- 12 broker architecture design 的上下文、优点、限制是什么? SOA、微服务都是 broker? ffffffffff
- 13 简要描述 the relationship between **architecture patterns** and **tactics** in architecture design.
(**patterns** and **tactics** 是第二阶段 architecture design 的输入) ffffffffff
- 14 attribute-driven design 的过程 process 是什么 ffffffffff
15. 计算宕机时间 (可用性) ffffffffff

历史遗留问题 11

8

1. 软件体系结构中有什么活动? 每个活动的输入输出?

Lecture1 p32 p33

这里强调"Architecture Process"

- 1.Specifying ASRs (识别 ASRs)
- 2.architecture design 架构设计
- 3.documenting 文档化
- 4.architecture evaluation 架构评估

输入输出

第一阶段输出: 优化的质量属性场景

第二阶段输入: 优化的质量属性场景、需求和约束、Patterns and tactics

第二阶段输出: 一组候选视图的草图 (由模式决定)

第三阶段输入: 一组候选视图的草图 (由模式决定)

第三阶段输出: Views and Beyond

第四阶段输入: Views and Beyond、优化的质量属性场景

第四阶段输出: Views and Beyond

2. what distinguishes an architecture for a software product line

from an architecture for a single product?

用什么 去区分 软件产品线体系结构

从 单一产品体系结构?

(乱答)

产品线是产品的集合, 产品=自定义资产+核心资产。

单一产品体系结构只有一个产品, 软件产品线体系结构有多个产品。

软件产品线体系结构能大程度地对现有资源进行重用。软件重用一直是软件工程界不断追求的目标。软件产品线是最大程度的软件重用, 可以有效地降低成本、缩短产品面世时间、提高软件质量。

单一产品体系结构对体系结构的变化没有说明和限制 ;必须在软件产品线体系结构中对允许进行的变化进行显式的说明和限定。

面向 独立软件系统的软件体系结构设计方法 并不完全适用于 软件产品线体系结构的设计, 因为它们一般没有考虑 产品线中 不同产品之间 的共性和个性问题。

(可以不写这段)产品线体系结构可以使 软件开发组织 将总成本均摊到 产品线的多个产品的设计开发中, 从而充分地降低整体成本、有效地提高软件生产率。

3. what are the generic design strategies applied in designing software?

Give a concise working example with software architecture for each strategy.

软件设计中通用的设计策略有哪些?

给出了每个策略的软件体系结构简洁的工作示例。

Lecture1 p26

- (1) 分解 (将整个系统分解或将某个模块分解)
- (2) 抽象 (将系统抽象为组件&连接件或抽象为模块)
- (3) 逐个划分击破 (将每个模块逐个击破)
- (4) 生成和测试(把一个特定的设计看作是一个假设 ; 根据测试路径生成测试用例)
- (5) 迭代:增量改进 (ADD 方法多次迭代直到满足所有 ASR)
- (6) 可重用元素 (重用已有架构等)

(1)的工作示例 : Lecture6 p4

(4)的工作示例 : Lecture6 p6

4. how to model quality attribute scenarios?

Graphically model the concrete scenarios of 2 qualities in "stimulus-response" format: availability and modifiability.

如何建模质量属性场景?

图形化地建模具体的情景, 两个质量属性用“刺激-响应”格式: 可用性和可修改性。(“刺激-响应”格式指的是那个 6 元素图)

Lecture2 3 p24

场景分为两类 :

a. 一般场景(general scenarios)是系统独立的场景，以指导质量属性需求的规范。(一般场景用一个 6 元素表描述)

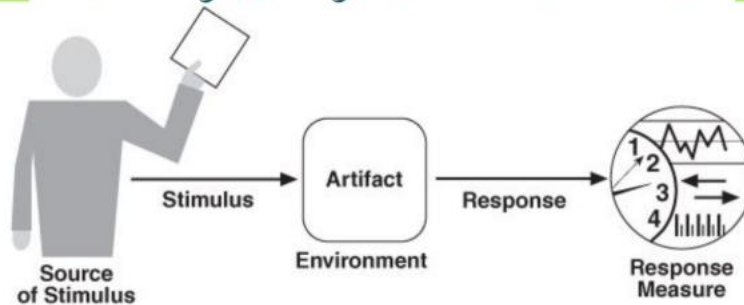
b. 具体场景(concrete scenarios)是系统特定的场景，以指导特定系统的质量属性需求规范。它们是一般情况的实例。(具体场景用一个 6 元素图描述)

正式答题：

Lecture2 3 p26

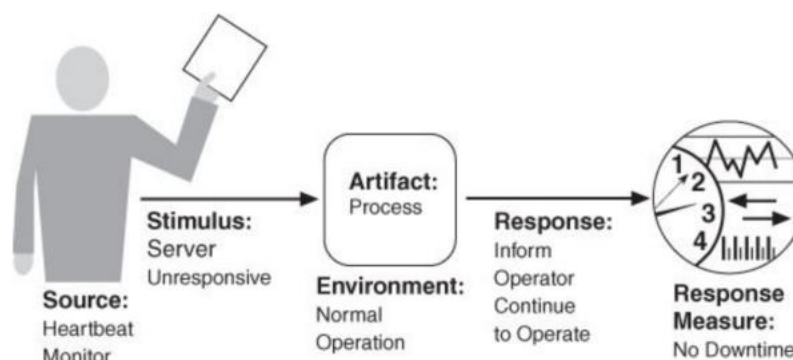
方法是使用 6 元素分析 (“刺激-响应”格式指的是这个 6 元素图)

Modeling Quality Attribute Scenarios

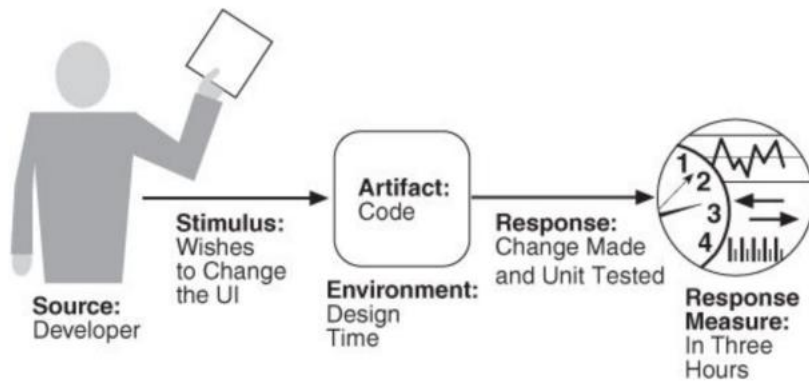


刺激	A condition that needs to be considered when it arrives at a system. 当它到达一个系统时需要考虑的条件。
刺激源	产生刺激的实体(人、系统或任何执行器)
制品	需求所应用的整个系统或系统的一部分
环境	当刺激发生时系统的状态。例如超载、正常运行等。
响应	刺激到达后进行的活動。
响应度量	对刺激的响应应该能以某些方式度量，以使需求能够被测试。

Availability Sample Scenario

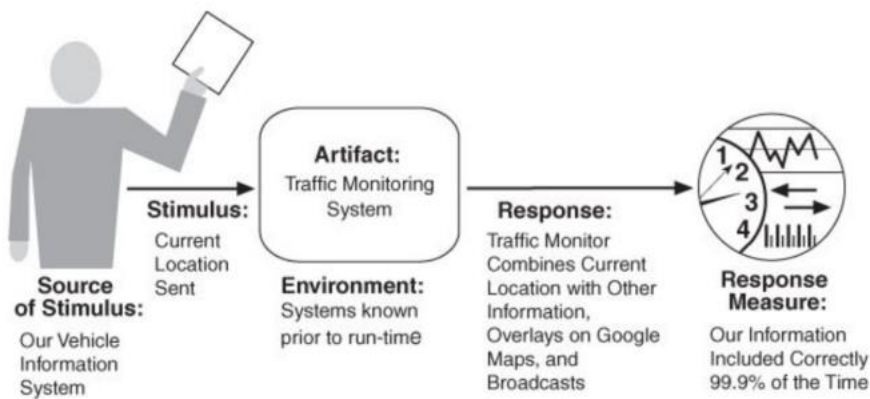


Modifiability *Sample Scenario*

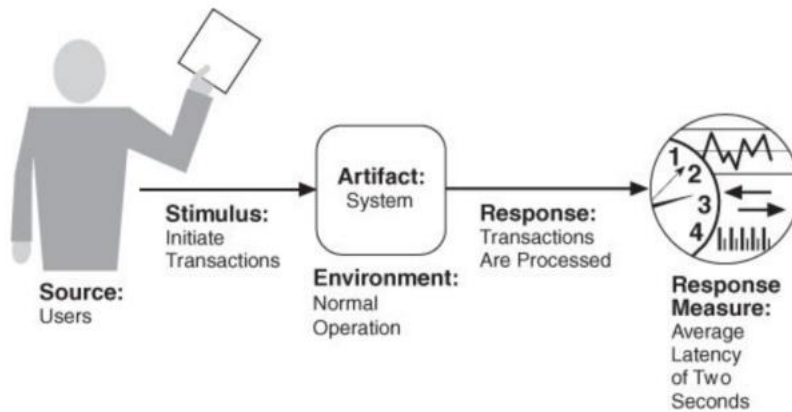


贴一下其他场景的：

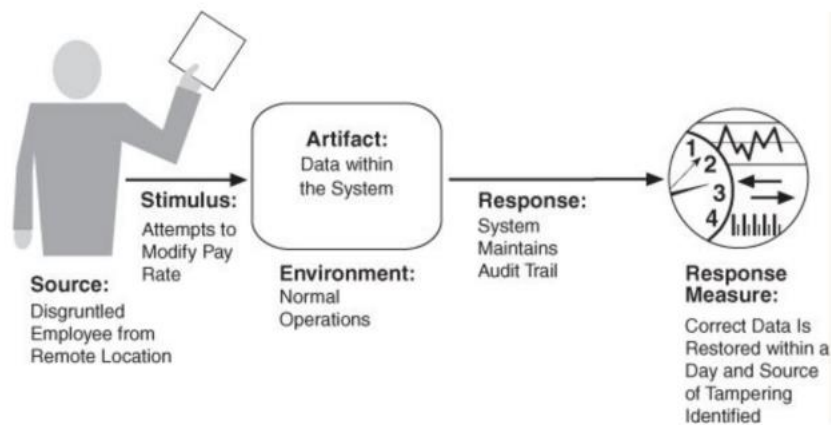
Interoperability *Sample Scenario*



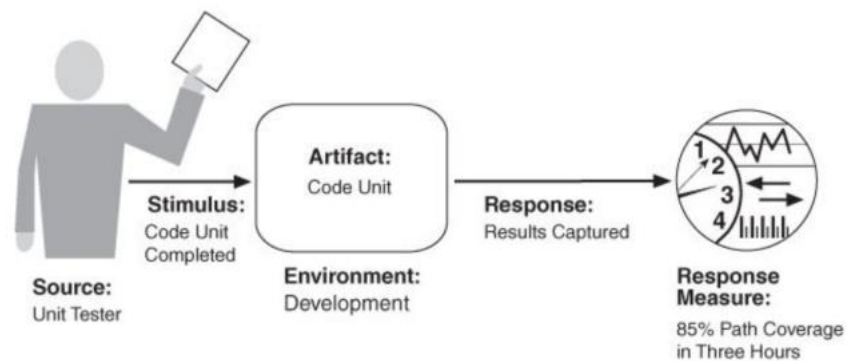
Performance Sample Scenario



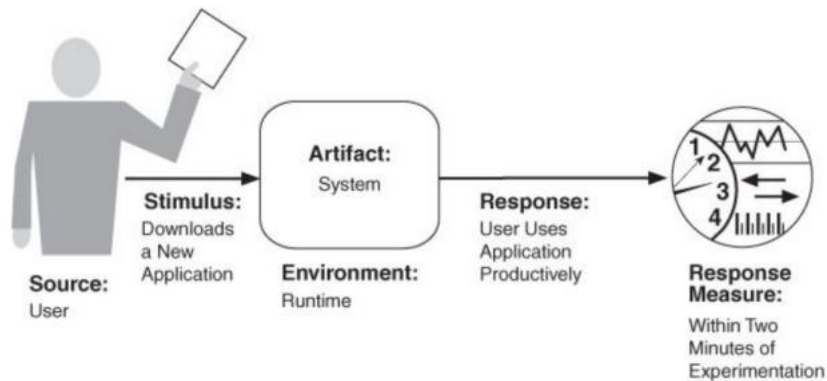
Security Sample Scenario



Testability Sample Scenario



Usability Sample Scenario



5. Describe the outputs generated from each phase of ATAM (architecture tradeoff analysis method) process.

描述那些输出，从 ATAM 过程的每个阶段生成的输出。

ATAM(架构权衡分析方法)

准备阶段：(1)评估计划

评估阶段：(1)架构的简洁展示

(2)业务目标的清晰度

(3)作为场景实现的 特定质量属性需求 的优先级列表。

(4)效用树

(5)风险和无风险决策

(6)敏感点和权衡点

再评估阶段：(1)来自涉众群体的优先场景列表

(2)受到威胁的风险主题和业务动机

Follow-up 阶段：(1)最终评估报告

6. 描述

组件-连接件风格

allocation styles 分配风格

module styles

Lecture7 p9

(1) module styles 认为体系结构是由模块组成。模块是实现单元的集合，它提供了一组一致的职责。

(2) C&C styles 认为体系结构是由 组件、连接件组成。

组件：主要的处理单元和数据存储。组件有一组端口，通过端口和其他组件交互（通过连接件）。

连接件：组件之间的交互路径。连接件有一组角色，角色表明组件之间在交互时如何使

用连接件。

(3) allocation styles 认为体系结构是由 **软件元素**、**环境元素** 组成。它展示了软件如何与环境关联。

软件元素：软件元素具有环境所需的属性。

环境元素：环境元素有提供给软件的属性。

7. 什么是 ASRs? ASRs 来自哪里?

List 4 sources or methods for extracting and identifying ASRs.

列出 4 来源或方法 去 提取与识别 ASRs.

Lecture 2 3 p110

包括质量属性, 重要的功能需求, 和约束

An Architecturally Significant Requirement (ASR) is a requirement that will have a profound effect on the architecture the architecture might well be dramatically different in the absence of such a requirement.

体系结构上的重要需求(ASR)是一种需求, 它将对体系结构产生深远的影响, 在缺乏这种需求的情况下, 体系结构可能会有很大的不同。

The more difficult and important the QA requirement, the more likely it is to significantly affect the architecture, and hence to be an ASR.

QA 越困难、越重要, 就越有可能显著影响体系结构, 从而成为 ASR。

How to systematically identify the ASRs and other factors that will shape the architecture?

Gathering ASRs from requirements documents.

Gathering ASRs by interviewing stakeholders.

Gathering ASRs by understanding the business goals.

Capturing ASRs in a utility tree.

如何系统地识别 会影响架构 的 ASRs 和其他因素?

从需求文档中收集 ASRs。

通过采访涉众来收集 ASRs。

通过了解业务目标来收集 ASRs。

在效用树中收集 ASRs。

8. 列出至少 3 个面向对象原则。解释它们在策略模式中怎么被利用(怎么体现)。

单一职责原则(Single Responsibility Principle):应该有且只有一个原因引起类的变更。
里氏替换原则(Liskov Substitution Principle):子类必须能替换掉父类而起同样的作用。
依赖倒置原则(Dependence Inversion Principle):抽象不应该依赖于细节, 细节应该依赖于抽象; 高层模块不应该依赖于低层模块, 而是双方都依赖于抽象。
接口分离原则(Interface Segregation Principle):接口应尽量小且高内聚。
迪米特法则(Law of Demeter):又叫最少知识原则, 一个类只和直接朋友说话, 不和陌生人说话, 不能出现 a.b.Method()这种情况。
开闭原则(Open Closed Principle):对修改关闭, 对扩展开放。

策略模式请上网查。

9. 什么该被包含, 在一个典型的体系结构文档中(即目录应该有啥)? 简要描述每一部分和它的目的。

Lecture7

应该有 Views and Beyond:

Views

styles and views(体系结构风格和视图)

structural views

module views(显示系统被分为哪些模块。模块是实现单元的集合, 它提供了一组一致的职责。)

C&C views(显示了一些运行时存在的元素, 如进程、对象、客户端、服务器)

allocation views(分配视图的通常目标是将 **软件元素**所要求的属性与 **环境元素**提供的属性进行比较, 以确定分配是否成功。)

quality views(质量属性视图, 如安 security view、performance view 等)

Beyond(除了视图以外的一些附加信息, 如

- 1.文档路标
- 2.视图是如何被文档化的
- 3.系统预览
- 4.视图间的映射
- 5.基本原理
- 6.目录——索引, 术语表, 缩略词表。

)

10. 描述 “4+1” Views

Lecture1 p27

Logical view: describes architecturally significant elements of the architecture and the

relationships between them.

Process view: describes the concurrency and communications elements of an architecture.

Physical view: depicts how the major processes and components are mapped on to the applications hardware.

Development view: captures the internal organization of the software components as held in e.g. a configuration management tool.

Architecture use cases: capture the requirements for the architecture; related to more than one particular view.

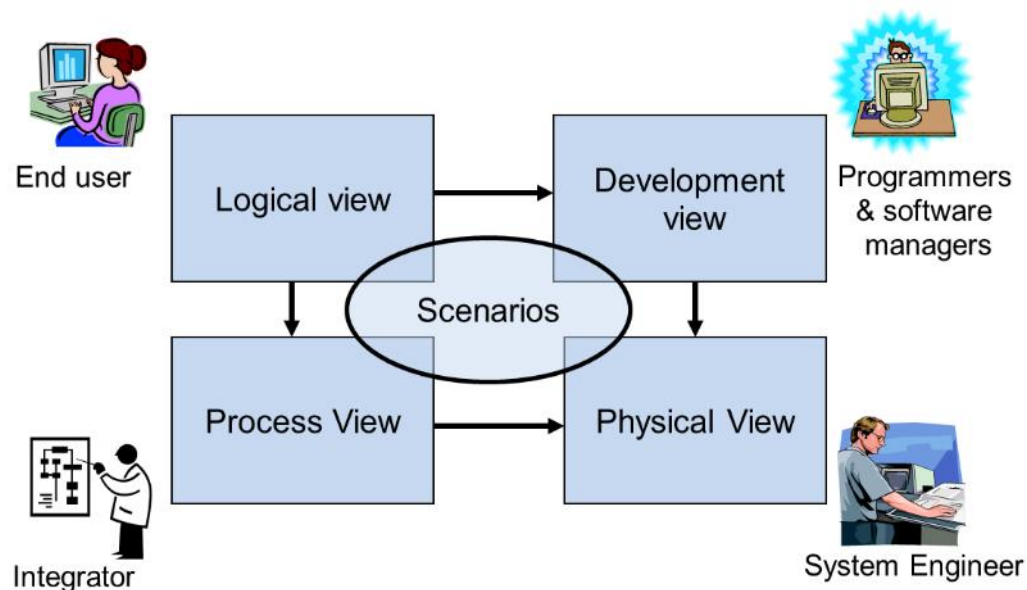
逻辑视图:描述体系结构的重要元素和它们之间的关系。

进程视图:描述架构的并发和通信元素。

物理视图:描绘主要进程和组件如何映射到应用程序硬件。

开发视图:捕获软件组件的内部组织, 例如配置管理工具。

体系结构用例:捕获体系结构的需求;与不止一个特定的观点有关。



11. what are the possible dimensions of variations in software design?

软件设计中变化的可能维度是什么?

Elaborate the possible variabilities in each dimension.

阐述 那些可能的变异 (在每个维度)。

Discuss the possible impacts of deferring binding time on different quality attributes, for example modifiability and testability.

讨论 那些 可能的 影响 of 推迟 绑定时间 在 不同的 质量属性, 例如可修改性和可测试性

Lecture 2 3 p62

考察可修改性

像作业 1task 2

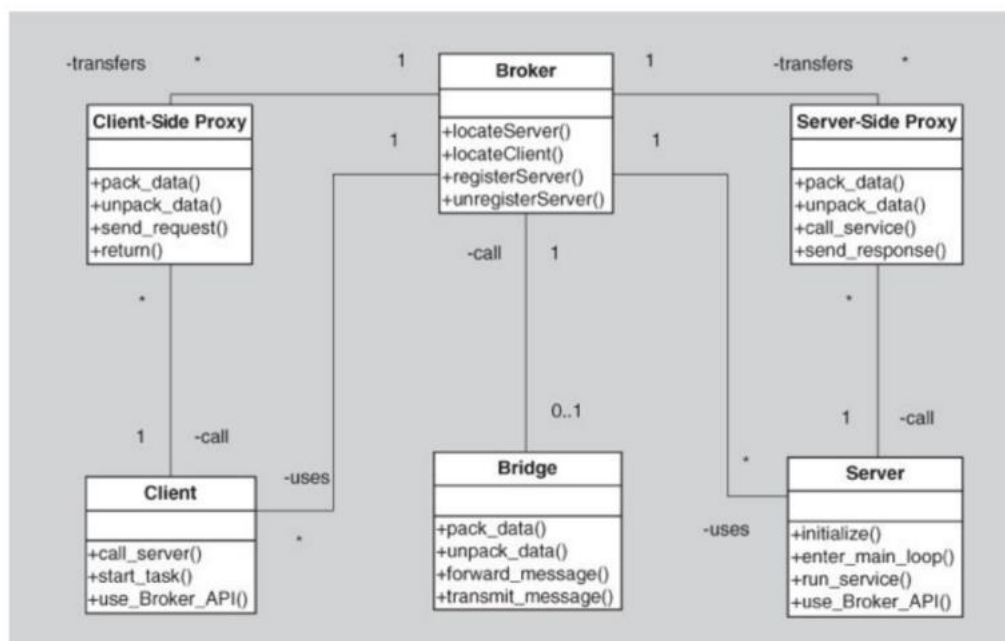
推迟绑定时间 是 第二个 pdf 中提高可修改性的策略。

推迟绑定时间：太早确定模块与模块之间的关系的话，后面越难修改，反之越容易修改。但是推迟绑定时间也会影响到其他质量属性。

12.broker architecture design 的上下文、优点、限制是什么？

SOA、微服务都是 broker?

Lecture 4 5 p18 p19



上下文：(预览) broker 模式定义了一个运行时组件，称为 broker，它将协调多个客户机和多个服务器之间的通信。当单体应用遇到性能瓶颈（如请求量大）时不妨考虑一下 broker 模式，用多台计算机来分担一下负担。

优点：

可以有多个 server，解决了单体应用的性能瓶颈。

提高可用性，即使一个 server 发生宕机，整个系统还是可用的。

降低客户端与服务器端的耦合。

允许动态地注册、注销 server。

限制：

客户端只能连接到 broker（可能通过客户端代理）。

服务器端只能连接到 broker（可能通过服务器端代理）。

由于客户端请求先要经过 broker 而不是直接到达服务器端，所以增加了延迟。

Broker 增加了预先的复杂性。
Broker 可能成为黑客攻击的目标。
Broker 使得测试难以进行。
Broker 发生宕机，整个系统不可用。

SOA 延续了 broker 模式的思想。
查找服务和使用服务时都要通过 broker ；
而 SOA 只在查找时通过 register（服务注册中心）；
SOA 分散了 broker 的职责，降低了 broker 的单点风险。

13. 简要描述 the relationship between **architecture patterns** and **tactics** in architecture design.

（**patterns** and **tactics** 是第二阶段 architecture design 的输入）

Lecture 4 5 p52

Tactics are simpler than patterns; they use a single structure or mechanism to address a single architectural force.

Patterns typically combine multiple design decisions into a package.

Patterns and tactics together constitute the software architect's primary tools.

Tactics are "building blocks" of design from which architectural patterns are created.

Most patterns consist of several different tactics that may:

- all serve a common purpose,
- be often chosen to promise different quality attributes.

Example: layered pattern

- Increase semantic coherence
- Restrict dependencies

- (a) 战术比模式简单；他们使用单一的结构或机制来处理单一的体系结构。
- (b) 模式通常将多个设计决策组合成一个包。
- (c) 模式和策略共同构成了软件架构师的主要工具。
- (d) 策略是设计的“构建块”，在此基础上创建架构模式。
- (e) 大多数模式由几种不同的策略组成：
 - 一切都有共同的目的，
 - 经常选择承诺不同的质量属性。

例如:分层模式

- 增加语义一致性
- 限制依赖关系

14. attribute-driven design 的过程 process 是什么？

Step 1: 确认有足够的需求信息

Step 2: 选择系统的一个元素来分解

Step 3: 识别所选元素的 ASRs。

Step 4: 选择一个满足 ASRs 的设计概念。

Step 4.1: 确定设计关注点。

Step 4.2: 为次要关注点列出可供选择的模式/策略。

Step 4.3: 从列表中选择模式/策略。

Step 4.4: 确定模式/策略和 ASRs 之间的关系。

Step 4.5: 捕获初步的架构视图

Step 4.6: 评估和解决矛盾

Step 5: 实例化架构元素并分配职责

Step 6: 为实例化元素定义接口

Step 7: 验证和精炼需求，并使它们成为实例化元素的约束。

Step 8: 重复，直到满足所有 ASRs。

15. 计算宕机时间（可用性）

Lecture 2 3 p35

可用性可以计算为它在指定的时间间隔内提供指定服务的概率。

MTBF 两次失效之间的时间

MTTR 维修时间

$$\text{Availability} = \text{MTBF} / (\text{MTBF} + \text{MTTR})$$

已知 ec2 可用性为 99.9%时一年 downtime 的时间是 8h0m46s

那么 ec2 可用性为 99.95%时一年 downtime 的时间是多少？

列出简单比例式计算即可

$$0.1\% / 0.05\% = 8\text{h}0\text{m}46\text{s} / x$$

解得 $x=4\text{h}0\text{m}23\text{s}$